Batocera v42 LED Indicator Setup

Setup & Troubleshooting Reference

Raspberry Pi 5 5 GPIO Pin (Physical) 1, 11, 13, 15, 29, 22, 25

Connector wiring - 9 wires total, 7 wires goes to pi

10 Pin FPC Breakout Pin	Pi 5 Physical Pin	Function
Pin 1 + Pin 2 Twist together and both together goes to Pi GPIO	Pin 1	3.3V
Pin 9 + Pin 10 Twist together and both together goes to Pi GPIO	Pin 25	GND
Pin 3 + 100 ohm Resistor	Pin 25	Col 2 LED 2 RED POST/Debug
Pin 4	Pin 13	Col 2 LED 3 WHITE POST/Debug
Pin 5	Pin 11	Col 1 LED 1 WHITE Network status
Pin 6	Pin 29	Col 4 LED 5 WHITE Temperature indicator
Pin 7	Pin 22	Col 3 LED 4 WHITE Storage/NVMe activity

Pin 8	Leave Unconnected	NC

LED Reference

LED Patterns and what means what

LED 1 White = Solid: internet | Blink: router only | Off: no network
LED 2 Red + LED 3 White = POST codes on boot, thermal warnings at runtime
LED 4 White = Flickers on NVMe read/write activity
LED 5 White = Solid when temperature below 60, slow blinking temp b/w 60 and 70, rapid blinking temp above 70

POST CODES (Red blinks · White blinks):

- R·W = NVMe missing
- R·WWW = No WiFi interface
- RR·W = Gamepad missing
- RR·WW = Thermal sensor gone
- RR·WW x1 = Temp warning 75C+
- RR·WW x3 = Temp critical 85C+

(Temperate codes and some other to be updated)

1. Batocera Configuration

1.1 Create the main service file:

Create this file on Batocera Alt + F5 or via SSH:

```
nano /userdata/system/scripts/led_service.py
```

Paste these code inside it:

```
#!/usr/bin/env python3
# led_service.py
# Full LED service for Raspberry Pi 5 handheld
# Batocera compatible, lgpio based

import lgpio
import time
import threading
import subprocess
import os

# ─────────────────────────────────────────────
# PIN DEFINITIONS (BCM numbers)
# ─────────────────────────────────────────────
PIN_NETWORK  = 17   # LED 1 White  - Network status
```

```
PIN_POST_RED = 22   # LED 2 Red    - POST/Debug/Battery
PIN_POST_WHT = 27   # LED 3 White  - POST/Debug/System
PIN_STORAGE  = 25   # LED 4 White  - NVMe activity
PIN_TEMPERATURE = 5    # LED 5 White  - Temperature Indication

ALL_PINS = [PIN_NETWORK, PIN_POST_RED, PIN_POST_WHT,
            PIN_STORAGE, PIN_TEMPERATURE]

running = True
h = None

# ──────────────────────────────────────────────
# CORE LED FUNCTIONS
# ──────────────────────────────────────────────
def led_on(pin):
    lgpio.gpio_write(h, pin, 0)

def led_off(pin):
    lgpio.gpio_write(h, pin, 1)

def all_off():
    for pin in ALL_PINS:
        led_off(pin)

def blink(pin, on_time=0.3, off_time=0.2, count=1):
    for _ in range(count):
        led_on(pin)
        time.sleep(on_time)
        led_off(pin)
        time.sleep(off_time)

# ──────────────────────────────────────────────
# POST BLINK CODES
# ──────────────────────────────────────────────
def post_code(red_count, white_count, repeat=3):
    for _ in range(repeat):
        for r in range(red_count):
            blink(PIN_POST_RED, 0.3, 0.15)
        time.sleep(0.4)
        for w in range(white_count):
            blink(PIN_POST_WHT, 0.3, 0.15)
        time.sleep(0.8)

# ──────────────────────────────────────────────
# POST HEALTH CHECKS
# ──────────────────────────────────────────────
def run_post_checks():
    """
```

```
    Blink codes:
    (1,1) = NVMe not found
    (1,3) = No network interface
    (2,1) = Gamepad not detected
    (2,2) = Thermal sensor missing
    """
    errors = []

    if not os.path.exists('/dev/nvme0n1'):
        print("[LED] POST: NVMe not found")
        errors.append((1, 1))
    else:
        print("[LED] POST: NVMe OK")

    if not os.path.exists('/sys/class/net/wlan0'):
        print("[LED] POST: No network interface")
        errors.append((1, 3))
    else:
        print("[LED] POST: Network interface OK")

    gamepad_found = os.path.exists('/dev/input/js0') or \
                    os.path.exists('/dev/input/js1')
    if not gamepad_found:
        print("[LED] POST: Gamepad not detected")
        errors.append((2, 1))
    else:
        print("[LED] POST: Gamepad OK")

    if not os.path.exists('/sys/class/thermal/thermal_zone0/temp'):
        print("[LED] POST: Thermal sensor missing")
        errors.append((2, 2))
    else:
        print("[LED] POST: Thermal sensor OK")

    return errors

# ──────────────────────────────────────────────
# BOOT SEQUENCE
# ──────────────────────────────────────────────
def boot_sequence():
    print("[LED] Boot sequence starting")

    start = time.time()
    while time.time() - start < 10:
        blink(PIN_POST_WHT, 0.5, 0.5, 1)

    errors = run_post_checks()
```

```
    if errors:
        for error_code in errors:
            print(f"[LED] POST fail: {error_code}")
            post_code(*error_code, repeat=4)
            time.sleep(1)

    all_off()
    time.sleep(0.5)

    print("[LED] Boot sequence complete")
    led_on(PIN_POST_WHT)

# ──────────────────────────────────────────
# THREAD 1 — NETWORK MONITOR
# ──────────────────────────────────────────
def check_network_state():
    try:
        if not os.path.exists('/sys/class/net/wlan0'):
            return 'none'

        result = subprocess.run(
            ['iwconfig', 'wlan0'],
            capture_output=True, text=True
        )
        if 'ESSID:off' in result.stdout or \
           'Not-Associated' in result.stdout:
            return 'none'

        ping = subprocess.run(
            ['ping', '-c', '1', '-W', '2', '8.8.8.8'],
            capture_output=True
        )
        return 'full' if ping.returncode == 0 else 'local'

    except Exception as e:
        print(f"[LED] Network check error: {e}")
        return 'none'

def network_monitor():
    print("[LED] Network monitor started")
    last_state = 'none'
    last_check = 0

    while running:
        now = time.time()

        if now - last_check >= 5:
            last_state = check_network_state()
```

```
            last_check = now
            print(f"[LED] Network state: {last_state}")

        if last_state == 'full':
            led_on(PIN_NETWORK)
            time.sleep(0.5)

        elif last_state == 'local':
            led_on(PIN_NETWORK)
            time.sleep(0.5)
            led_off(PIN_NETWORK)
            time.sleep(0.5)

        else:
            led_off(PIN_NETWORK)
            time.sleep(0.5)

# ──────────────────────────────────────────
# THREAD 2 — STORAGE ACTIVITY MONITOR
# ──────────────────────────────────────────
def storage_monitor():
    print("[LED] Storage monitor started")
    prev_count = 0

    while running:
        try:
            if os.path.exists('/sys/block/nvme0n1/stat'):
                with open('/sys/block/nvme0n1/stat') as f:
                    stats = f.read().split()
                current_count = int(stats[0]) + int(stats[4])

                if current_count != prev_count:
                    led_on(PIN_STORAGE)
                    prev_count = current_count
                else:
                    led_off(PIN_STORAGE)
            else:
                led_off(PIN_STORAGE)

        except Exception as e:
            print(f"[LED] Storage monitor error: {e}")
            led_off(PIN_STORAGE)

        time.sleep(0.15)

# ──────────────────────────────────────────
# THREAD 3 — TEMPERATURE LED (LED 5)
# ──────────────────────────────────────────
```

```
def temp_led_monitor():
    """
    LED 5 White temperature indicator:
    Solid        = below 60°C  (cool)
    Slow blink   = 60-72°C     (normal load)
    Fast blink   = 72-82°C     (heavy load)
    Rapid blink  = above 82°C  (thermal warning)
    """
    print("[LED] Temp LED monitor started")

    TEMP_COOL   = 60
    TEMP_WARM   = 72
    TEMP_HOT    = 82

    while running:
        try:
            with open('/sys/class/thermal/thermal_zone0/temp') as f:
                temp = int(f.read()) / 1000

            if temp < TEMP_COOL:
                led_on(PIN_TEMPERATURE)
                time.sleep(1)

            elif temp < TEMP_WARM:
                led_on(PIN_TEMPERATURE)
                time.sleep(0.8)
                led_off(PIN_TEMPERATURE)
                time.sleep(0.8)

            elif temp < TEMP_HOT:
                led_on(PIN_TEMPERATURE)
                time.sleep(0.25)
                led_off(PIN_TEMPERATURE)
                time.sleep(0.25)

            else:
                led_on(PIN_TEMPERATURE)
                time.sleep(0.08)
                led_off(PIN_TEMPERATURE)
                time.sleep(0.08)

        except Exception as e:
            print(f"[LED] Temp LED error: {e}")
            led_off(PIN_TEMPERATURE)
            time.sleep(1)

# ---------------------------------------------
# MAIN
```

```
# ──────────────────────────────────────────
def main():
    global running, h

    print("[LED] Opening GPIO handle")
    h = lgpio.gpiochip_open(0)

    print("[LED] Initializing pins")
    for pin in ALL_PINS:
        lgpio.gpio_claim_output(h, pin, 1)

    time.sleep(1)

    try:
        boot_sequence()

        threads = [
            threading.Thread(
                target=network_monitor,
                daemon=True,
                name='network'
            ),
            threading.Thread(
                target=storage_monitor,
                daemon=True,
                name='storage'
            ),
            threading.Thread(
                target=temp_led_monitor,
                daemon=True,
                name='temperature'
            ),
        ]

        for t in threads:
            t.start()
            print(f"[LED] Thread started: {t.name}")

        while True:
            time.sleep(1)

    except KeyboardInterrupt:
        print("\n[LED] Shutting down")
        running = False
        time.sleep(1)

    finally:
        all_off()
```

```
        lgpio.gpiochip_close(h)
        print("[LED] Cleanup done")

if __name__ == '__main__':
    main()
```

Save with Ctrl+X → Y → Enter

1.2 Use `custom.sh` for script auto start:

Use `custom.sh` which is Batocera's official autostart hook:

```
nano /userdata/system/custom.sh
```

Add this to the bottom:

```
bash /userdata/system/scripts/led_launcher.sh &
```

Full file should look like:

```
#!/bin/bash
modprobe fuse
bash "/userdata/system/configs/firefox/restore_desktop_entry.sh" &
python3 /userdata/system/scripts/volume_buttons.py &
python3 /userdata/system/scripts/fan_pwm.py &
python3 /userdata/system/scripts/led_service.py &
```

Save with Ctrl+X → Y → Enter

Verify all files are in place

```
ls /userdata/system/scripts/
```

Should show below files:

```
led_service.py
```

1.4 Test manually before rebooting:

Start script:

```
python3 /userdata/system/scripts/led_service.py &
```

Expected behavior:

```
Detects EmulationStation immediately
Starts LED service
LED 3 white pulses slowly for 10 seconds
POST checks run, all should pass
LED 3 goes solid white
```

```
LED 1 reflects network state
LED 4 flickers on storage activity
LED 5 turns on when a game launches
```

Stop script and kill process, check running processes:

```
ps aux | grep led
```

Kill the process which says led_launcher or led_service using it's PID:

```
kill PID
```

Check running processes:

```
ps aux | grep led
```

The process should have stopped.

Reboot:

```
reboot
```

After reboot check to see if script auto started or not

```
ps aux | grep led
```

2. Problems Faced

- First of all, the LEDs turn ON when GND is pulled out instead of normal current pushing. It took a while to figure out that we have to provide the LED board with 3.3V and GND at the same time, then pull GND from specific pins for the LED to glow. I had to look into laptop schematics for help.
- Was checking which LEDs glow by giving single pin 3.3V and a single pin GND, so pin configuration was different, when connected to pi and gave Pin 1 and 2 power and 9 and 10 GND, pin config changed, causing RED LED to not glow, probed and tested again to find the new configuration.
- Earlier I'd thought to use LED 5 for Audio ON/Mute/Earphone Jack IN and the code for that script worked correctly too (apart from the headphone jack detection which I skipped after two three tries as it wasn't that important). But when I added a script to autostart when Pi boots up, the audio driver/device was not getting detected, it was showing Red-White-White error code (meaning audio device not detected) and the LED 5 wasn't getting turned ON. But the audio was working and I was able to hear the music. I tried everything, from delaying the script running, to starting script when EmuStation boots up, checking for processes if GPIO pin is already claimed by another process and many more for about 3 to 4 hours till 2 midnight, but no luck, it didn't work, then out of tiredness and not wanting to end the day on a half done module, I ditched the Audio LED and thought to add some other indication for the LED 5.
- Which brings us to Emulation detection, which is good I guess, it'll let us know when something is loading or if the OS is crashed/freezed, in conjunction with the NVMe

LED. Implemented that and for now atleast the LED module is complete, will have to change the Debug Indicator Patterns and the Voltage Indication too.

- The Emulator LED was not of any use, as we can guess if something is operating or not just by looking at the NVMe LED. So switched it to a Temperature Indicator, solid when below 60 degrees, slow blinking b/w 60-70, rapid blinking.
